Projet Numérique Durable
BiblioLibre

Projet réalisé par :
Lilia Aourou, Timothée Boisseaux, Rayane Boussoura, Enzo Caçote, Louis Cardoso

1.Présentation du projet

BiblioLibre est une plateforme web qui permet aux étudiants de déposer et de consulter des ressources pédagogiques (fiches de révision, résumés de cours, annales) sous forme de liens URL. L'idée de départ est simple : les supports de cours circulent habituellement par e-mail, groupes WhatsApp ou drives personnels sans structure commune. BiblioLibre centralise ces échanges dans une interface légère, accessible sans inscription pour la consultation.

Le parti pris Green IT est appliqué dès la conception : aucun fichier n'est hébergé sur le serveur, uniquement des liens. La plateforme ne charge aucune dépendance JavaScript externe et utilise la police système du navigateur, ce qui ramène le poids d'une page à moins de 200 Ko.

Utilisateurs cibles

Trois profils d'utilisateurs ont été identifiés. Le visiteur non inscrit peut consulter et filtrer les ressources sans créer de compte, c'est la fonctionnalité la plus utilisée et elle ne nécessite aucune authentification. L'étudiant inscrit peut en plus déposer des ressources, les modifier et les supprimer. L'administrateur dispose d'un accès complet à la gestion des comptes et des contenus.

Périmètre du MVP

La version livrée couvre la gestion complète des utilisateurs (création, modification, suppression avec confirmation), le dépôt et la consultation de ressources avec filtrage par matière et promotion, et une interface d'administration. Les fonctionnalités écartées pour ce premier livrable sont la notation des ressources, la recherche plein-texte avancée et les notifications par e-mail.

Contrainte	Mesure concrète
Poids de page < 200 Ko	1 fichier CSS, 0 JS externe, police système
Requêtes HTTP < 10 par page	Pas de CDN externe, pas d'image décorative
Aucun fichier binaire stocké	Seuls les liens URL sont enregistrés en base
Filtres sans JavaScript	Formulaires HTML natifs avec méthode GET
Base de données minimale	Schéma 3NF, index sur les colonnes interrogées

2.Architecture et conception

Le projet suit une architecture MVC classique avec Flask. Les routes sont organisées en blueprints indépendants, chacun gérant un périmètre fonctionnel précis. La base de données SQLite est accédée via des requêtes SQL paramétrées directes, sans ORM, ce qui limite les dépendances et conserve la lisibilité des requêtes

Couche	Technologie	Rôle
Front-end	HTML5 + CSS3	Templates Jinja2, rendu côté serveur
Back-end	Python / Flask 3	Blueprints : auth, ressources, admin, profil
Base de données	SQLite	3 tables : utilisateur, ressource, matiere
Déploiement	Render.com	WSGI Gunicorn, HTTPS automatique

Diagramme de cas d'utilisation

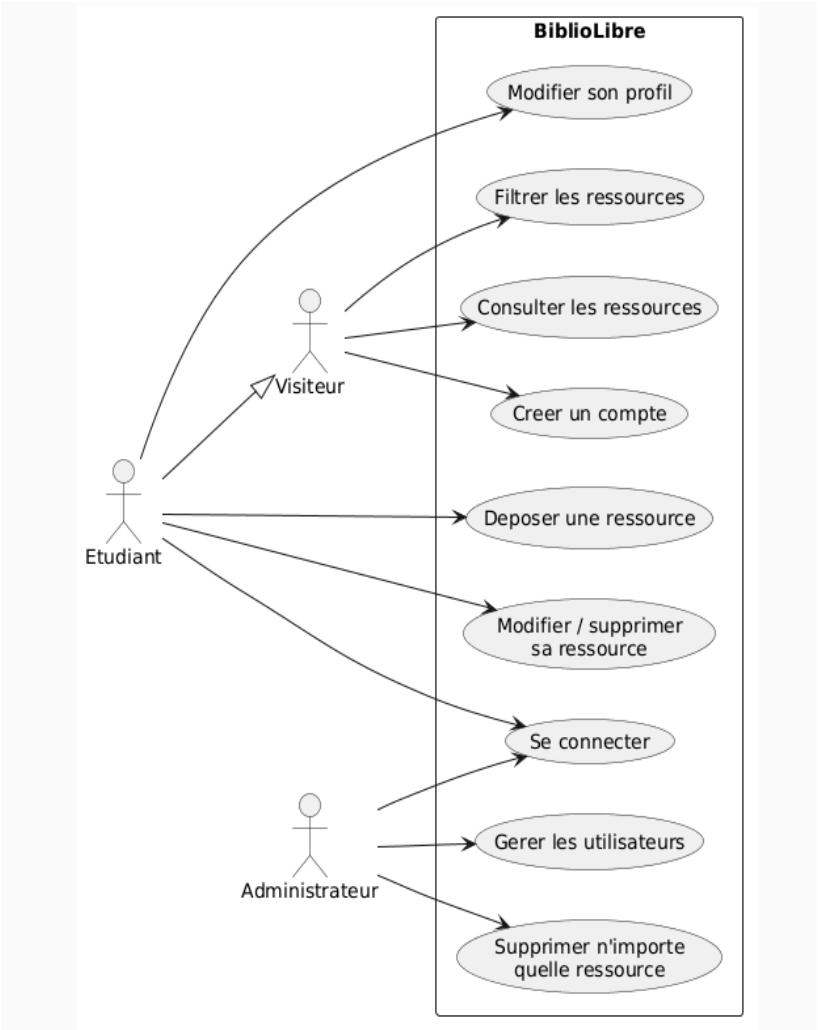


Diagramme de classe

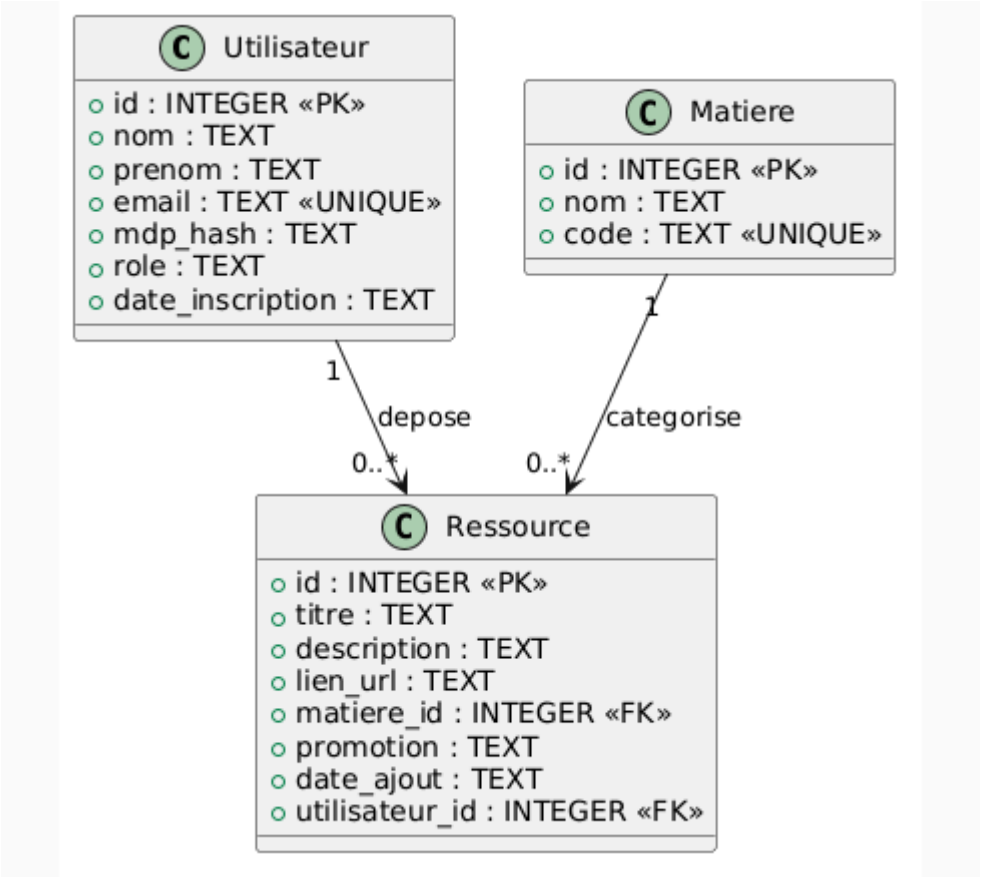


Diagramme de séquence connexion d'un étudiant

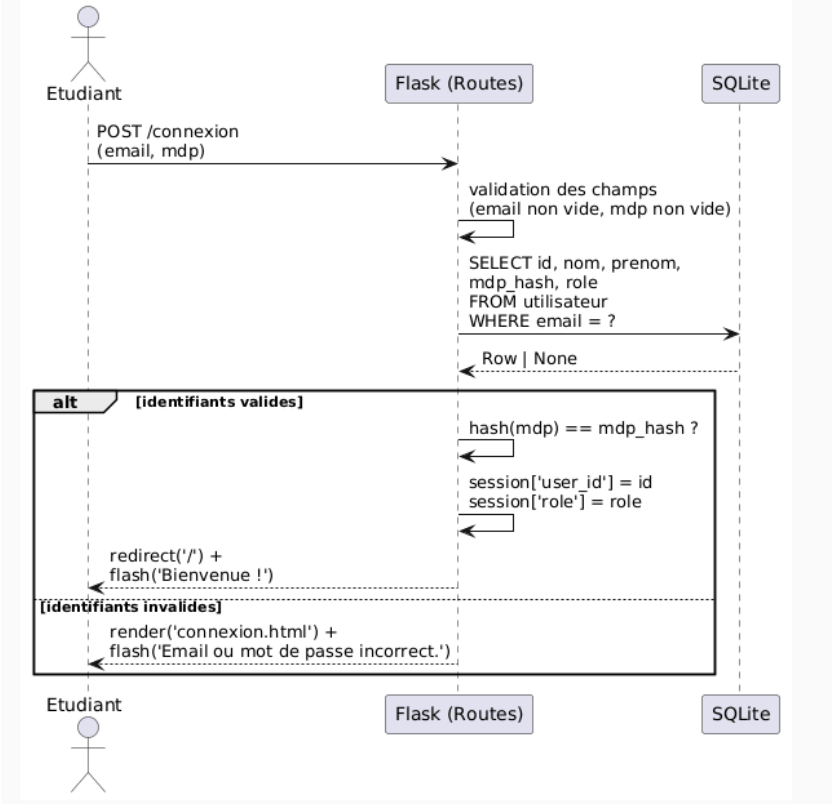


Schéma de la base de données

La base comporte trois tables. utilisateur stocke les informations de compte avec le mot de passe hashé en SHA-256. matière normalise les catégories de ressources (évite les doublons textuels). ressource relie un étudiant à un lien URL avec ses métadonnées. La clé étrangère utilisateur_id est définie avec ON DELETE CASCADE : supprimer un compte supprime automatiquement toutes ses ressources.

utilisateur

```
id               INTEGER PRIMARY KEY AUTOINCREMENT,
nom              TEXT    NOT NULL,
prenom           TEXT    NOT NULL,
email            TEXT    UNIQUE NOT NULL,
mdp_hash         TEXT    NOT NULL,
role             TEXT    NOT NULL DEFAULT 'user',
date_inscription TEXT    NOT NULL
```

matiere

```
id   INTEGER PRIMARY KEY AUTOINCREMENT,
nom  TEXT NOT NULL,
code TEXT NOT NULL UNIQUE
```

ressources

```
id             INTEGER PRIMARY KEY AUTOINCREMENT,
titre          TEXT    NOT NULL,
description    TEXT,
lien_url       TEXT    NOT NULL,
matiere_id     INTEGER REFERENCES matiere(id) ON DELETE SET NULL,
promotion      TEXT,
date_ajout     TEXT    NOT NULL,
utilisateur_id INTEGER NOT NULL REFERENCES utilisateur(id) ON DELETE CASCADE
```

3.Choix des technologies justifiés

Composant	Choix	Justification Green IT
Front-end	HTML5 + CSS3 natif	Zéro framework JS. Un seul fichier CSS de 8 Ko. Police système (system-ui)
Back-end	Python / Flask 3	Micro-framework minimaliste. 2 dépendances uniquement (flask, werkzeug). Temps de démarrage < 1 s.

Base de données	SQLite	Aucun serveur dédié. Fichier unique, empreinte mémoire quasi nulle. Parfaitement adapté au volume attendu.
Hébergement	Render.com	Plan gratuit avec HTTPS automatique. Serveur mutualisé. Déploiement déclenché automatiquement depuis main.
Polices	system-ui (CSS)	La police est celle du système de l'utilisateur, zéro octet téléchargé, rendu natif
Sessions	flask.session	Pas de base de données de sessions externe. Légèreté maximale côté serveur.

4.Implémentation

Le code est organisé en blueprints Flask séparés, un par domaine fonctionnel. Cette séparation a facilité la répartition du travail entre les membres de l'équipe et permet à chacun de travailler sur son périmètre sans modifier les fichiers des autres.

Fichier	Répartition	Contenu
app.py	Timothée	Factory Flask, route index, connexion BDD
init_db.py + database/schema.sql	Timothée	Initialisation et schéma de la base de données
routes/auth.py	Enzo	Inscription, connexion, déconnexion
routes/ressources.py	Louis	CRUD ressources, filtrage, pagination
routes/admin.py + profil.py	Rayane	Administration utilisateurs, profil
static/style.css + templates/base.html	Lilia	Design, layout, templates partagés

CRUD utilisateurs
La création de compte vérifie l'unicité de l'email, valide le format, contrôle la longueur du mot de passe (8 caractères minimum) et stocke uniquement le hash SHA-256. La liste des utilisateurs dans l'interface admin est chargée avec un COUNT des ressources associées en une seule requête jointure pour éviter les N+1 queries.

CRUD ressources
La liste des ressources utilise un système de filtres GET natifs (sans JavaScript) et une pagination côté serveur avec LIMIT/OFFSET. Seules les colonnes nécessaires sont sélectionnées donc pas de SELECT *. Un index est défini sur matiere_id, utilisateur_id et date_ajout pour accélérer les filtres les plus courants.

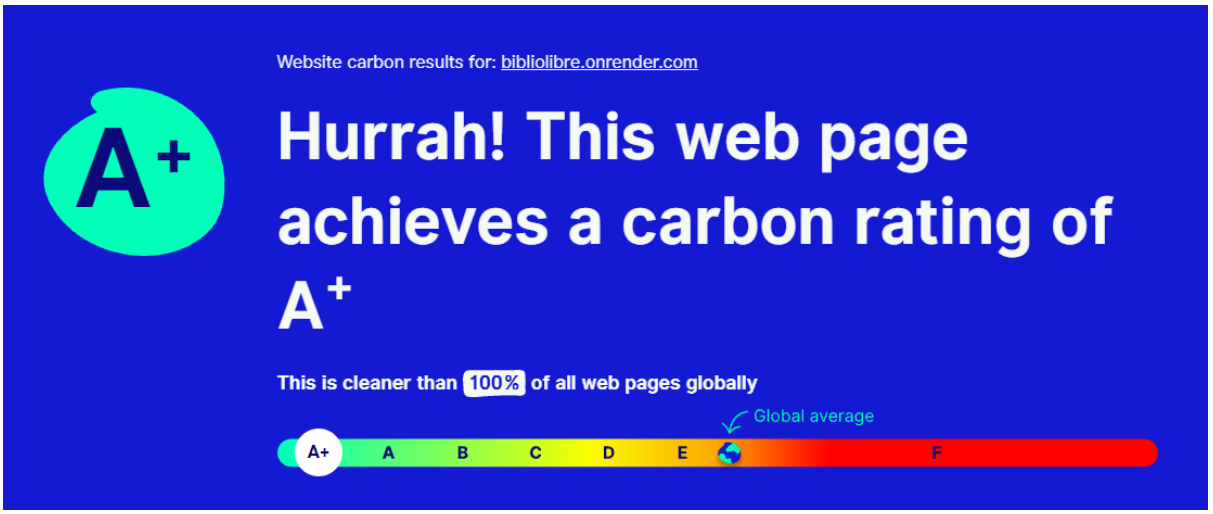
Sécurité
Toutes les requêtes SQL utilisent des paramètres positionnels, la concaténation de chaînes est absente du code. Les routes sensibles sont protégées par les décorateurs @login_required et @admin_required définis dans routes/helpers.py. Un utilisateur non connecté qui tente d'accéder à une URL protégée est redirigé vers la page de connexion. Les droits sont vérifiés côté serveur avant chaque opération de modification ou suppression

5.Analyse de l'empreinte carbone

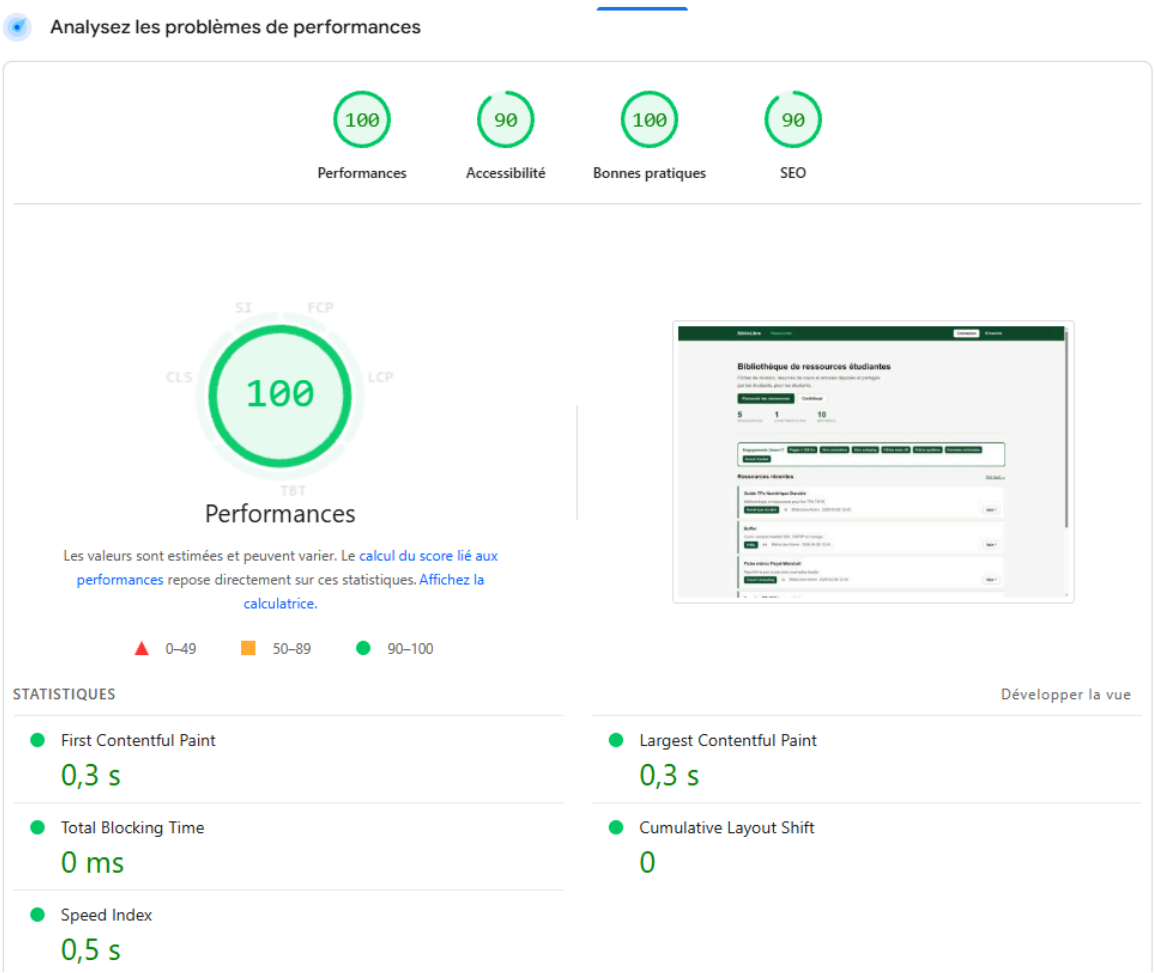
Démarche
La démarche que nous avons adoptée consiste à faire les choix techniques guidés par des contraintes environnementales dès la phase de conception, ce qui rend un tableau avant/après peu pertinent : il n'y a pas eu d'état initial dégradé à corriger. Cette section documente les mesures réelles du site et les compare à des références du même type pour quantifier les bénéfices des choix réalisés.

Outils de mesure utilisés
L'analyse a été conduite avec trois outils : EcoIndex (note A à G basée sur le poids de page, le nombre de requêtes HTTP et la complexité du DOM), Website Carbon Calculator (estimation de l'empreinte CO2 par visite) et Google Lighthouse (scores Performance, Accessibilité, Bonnes pratiques, SEO). Les mesures ont été réalisées sur la page d'accueil, la liste des ressources et la page de connexion.

Rapport EcodIndex



Rapport Website Carbon Calculator



Rapport Google Lighthouse

Analyse des résultats

Les résultats obtenus confirment la pertinence de l'approche que nous avons adoptée pour BiblioLibre. Sur Lighthouse, le site obtient un score de performance de 100/100, avec un First Contentful Paint de 0,3 s, un Largest Contentful Paint de 0,3 s, un Total Blocking Time de 0 ms et un Cumulative Layout Shift de 0, soit des métriques au niveau maximum sur tous les indicateurs Core Web Vitals. Les scores Accessibilité (90/100), Bonnes pratiques (100/100) et SEO (90/100) complètent ce tableau.

Sur EcoIndex, la page d'accueil obtient un score de 92/100 et une note A, avec seulement 2 requêtes HTTP, un poids de page de 0,005 Mo (5 Ko) et 98 éléments DOM. Ce résultat place BiblioLibre dans le top 8 591 des 595 851 pages analysées par l'outil.

Website Carbon attribue la note A+, indiquant que la page est plus sobre que 100 % des pages web analysées dans le monde.

Ces résultats s'expliquent directement par les choix techniques retenus : l'absence de framework JavaScript, l'utilisation de la police système, le rendu côté serveur avec Jinja2 et la limitation stricte des ressources statiques à un seul fichier CSS. La page entière pèse moins de 5 Ko, ce qui la place dans une catégorie exceptionnelle à l'échelle du web mondial.

6. Tests et validation

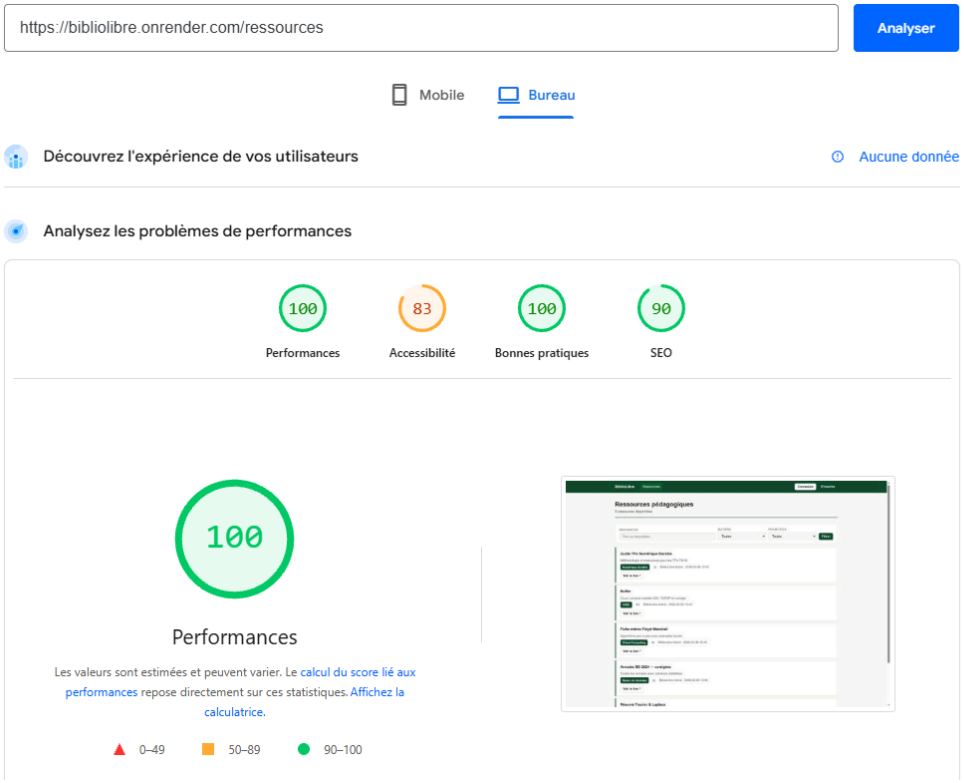
Tests fonctionnels

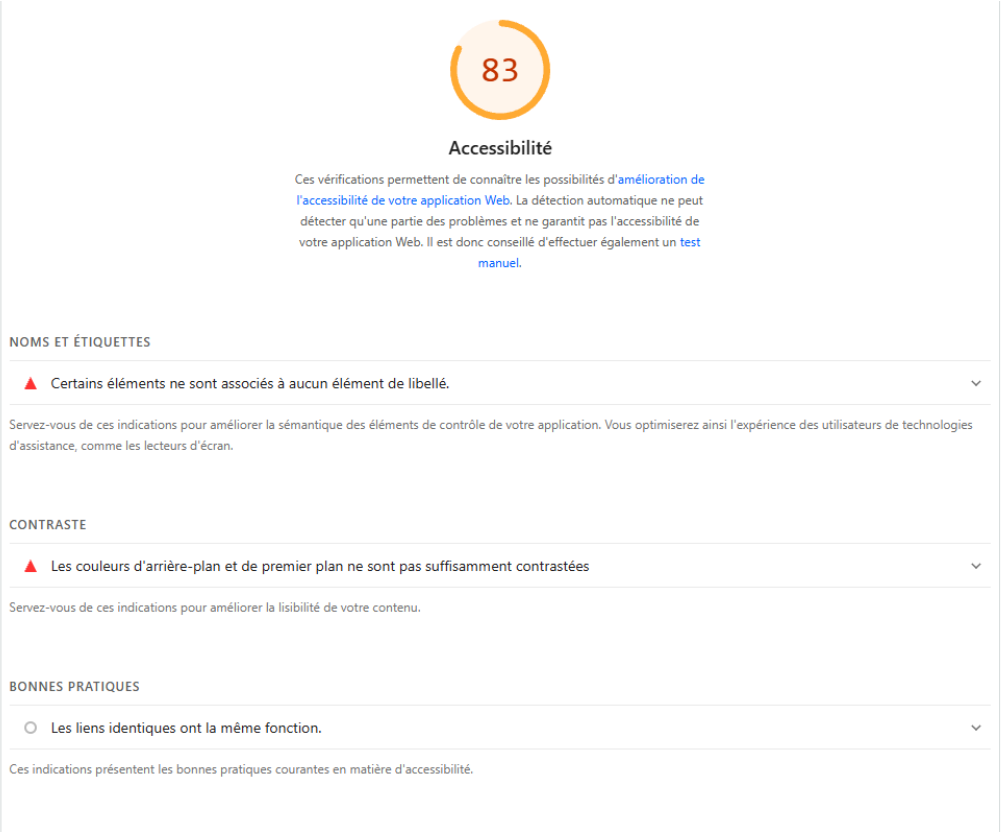
Les scénarios ci-dessous ont été exécutés manuellement sur le site déployé. Les résultats sont consignés avec le statut OK (fonctionnel), KO (défaillant) ou NT (non testé).

Scénario	Résultat attendu	Statut
Créer un compte avec des données valides	Compte créé en BDD, redirection vers connexion	OK
Créer un compte avec email déjà utilisé	Message d'erreur, aucune insertion en BDD	OK
Créer un compte avec mot de passe < 8 caractères	Message d'erreur côté serveur	OK
Se connecter avec identifiants valides	Session créée, redirection vers l'accueil	OK
Se connecter avec mauvais mot de passe	Message d'erreur, aucune session créée	OK
Accéder à /profil sans être connecté	Redirection vers /connexion	OK
Accéder à /admin sans être admin	Message d'erreur, redirection vers l'accueil	OK
Modifier une ressource d'un autre utilisateur (URL directe)	Accès refusé, message d'erreur	OK
Supprimer un utilisateur en tant qu'admin	Suppression + ressources associées supprimées (CASCADE)	OK
Filtrer les ressources par matière	Liste filtrée, pagination correcte	OK

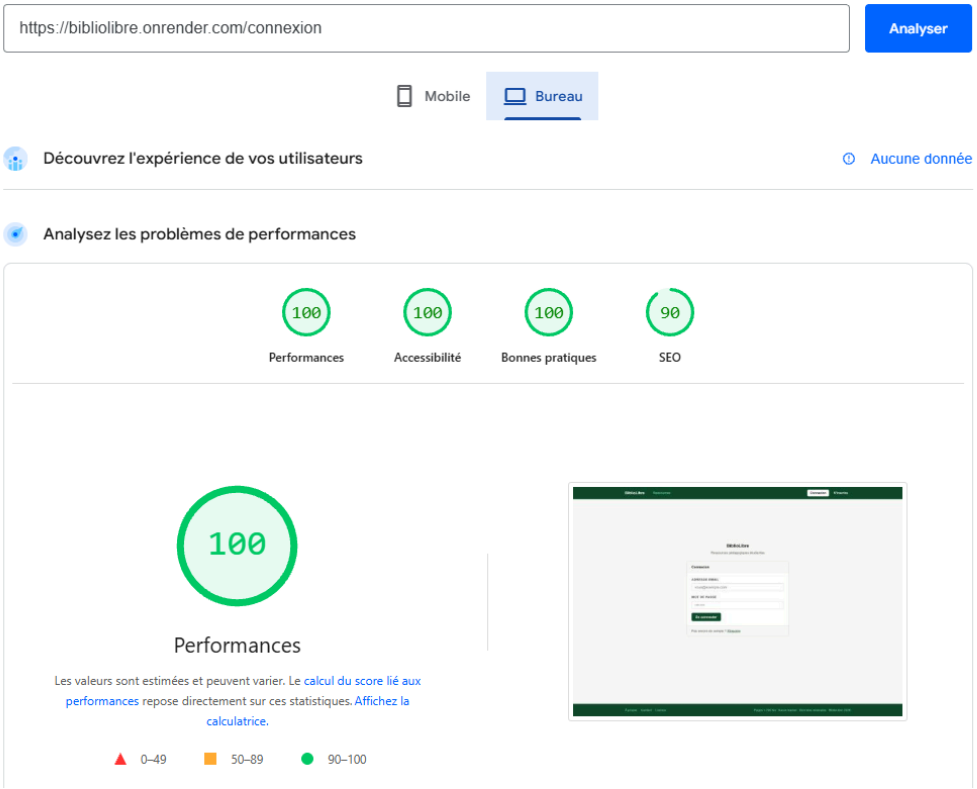
Tests de performance avec Google Lighthouse
Lighthouse a été exécuté sur les trois pages principales (accueil, ressources et connexion).
La page accueil à déjà été testé dans le section précedente

Page ressources :





Page connexion :



Page	Performance	Accessibilité	Bonnes pratiques	SEO
Accueil	100	90	100	90
Ressources	100	83	100	90
Connexion	100	100	100	90

Analyse des résultats

Performance 100/100 sur les trois pages. Bonnes pratiques 100/100 partout. C'est le résultat direct de l'absence de JS externe, de la police système et du CSS minimaliste.

Cependant l'accessibilité de la page /ressources tombe à 83. Lighthouse identifie deux problèmes concrets : certains éléments de formulaire (les selects des filtres) n'ont pas de <label> associé correctement, et le contraste entre certaines couleurs de texte et le fond n'est pas suffisant. Le troisième point "liens identiques avec la même fonction" concerne probablement les boutons "Voir" répétés sur chaque ressource.

Dans l'ensemble, les résultats Lighthouse valident les choix de conception et attestent qu'un site sobre n'est pas synonyme de site dégradé.

7. Organisation de l'équipe et collaboration

Répartition des tâches

Membre	Rôle	Fichiers principaux
Timothée Boisseaux	Chef de projet, architecture	app.py, init_db.py, database/schema.sql, routes/helpers.py, README.md
Enzo Caçote	Back-end authentification	routes/auth.py, templates/auth/
Louis Cardoso	Back-end ressources	routes/ressources.py, templates/ressources/
Lilia Aourou	Front-end, design, CSS	static/style.css, templates/base.html, templates/index.html
Rayane Boussoura	Back-end admin et profil	routes/admin.py, routes/profil.py, templates/admin/, templates/profil.html

Chaque membre a travaillé sur une branche dédiée et a ouvert une Pull Request vers main. Les merges ont été effectués dans un ordre précis pour éviter les conflits : architecture en premier, puis le CSS et les templates de base, puis les blueprints auth, ressources et enfin admin. Les Issues GitHub ont été utilisées pour suivre les tâches du backlog.

8. Discussion et conclusion

Défis rencontrés

La principale difficulté technique a été la gestion des droits dans les templates Jinja2 : afficher ou masquer les boutons Modifier et Supprimer selon que l'utilisateur est l'auteur d'une ressource ou un administrateur a nécessité de passer les bonnes variables de session dans le context_processor. La coordination entre les branches Git a aussi demandé de la rigueur, notamment pour respecter l'ordre de merge et éviter les conflits sur les fichiers partagés comme base.html.

Compromis entre fonctionnalités et sobriété

Le choix d'exclure tout framework JavaScript est le compromis le plus structurant du projet. Il simplifie considérablement le déploiement et réduit le poids des pages, mais impose que toutes les interactions (filtres, pagination, suppression) passent par des rechargements de page. Ce choix est cohérent avec la cible (ressources documentaires consultées plutôt qu'interactives) et avec les contraintes Green IT, mais il serait discutable pour une application nécessitant des mises à jour en temps réel.

Le stockage de liens URL plutôt que de fichiers est une contrainte acceptée dès le départ. Elle élimine les problèmes de stockage serveur, de gestion des formats de fichiers et de droits d'auteur, mais transfère la responsabilité de disponibilité du lien à l'utilisateur qui le dépose.

Pistes d'amélioration

Plusieurs fonctionnalités méritent d'être développées dans une version 2 : un système de recherche plein-texte côté serveur (avec FTS5 de SQLite), le hashage des mots de passe avec bcrypt plutôt que SHA-256, et un mécanisme de signalement des liens morts. Sur le plan de la performance, la mise en cache HTTP des ressources statiques avec des en-têtes Cache-Control serait un gain immédiat sans complexité supplémentaire.

Conclusion

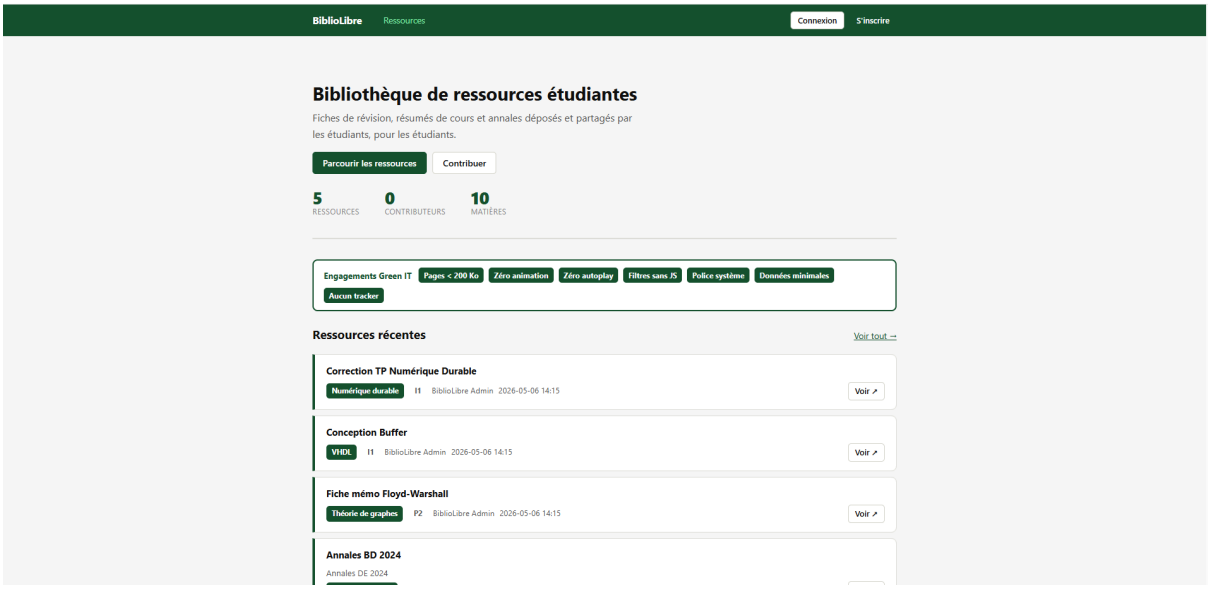
BiblioLibre atteint les objectifs fixés pour ce mini-projet : une application web fonctionnelle avec un CRUD complet sur deux entités, une interface sobre sans dépendance JavaScript, et une démarche Green IT documentée et mesurable. Le fait de travailler en équipe de cinq sur un dépôt Git partagé a aussi été formateur sur le plan de la coordination et de la rigueur de code.

9.Annexes

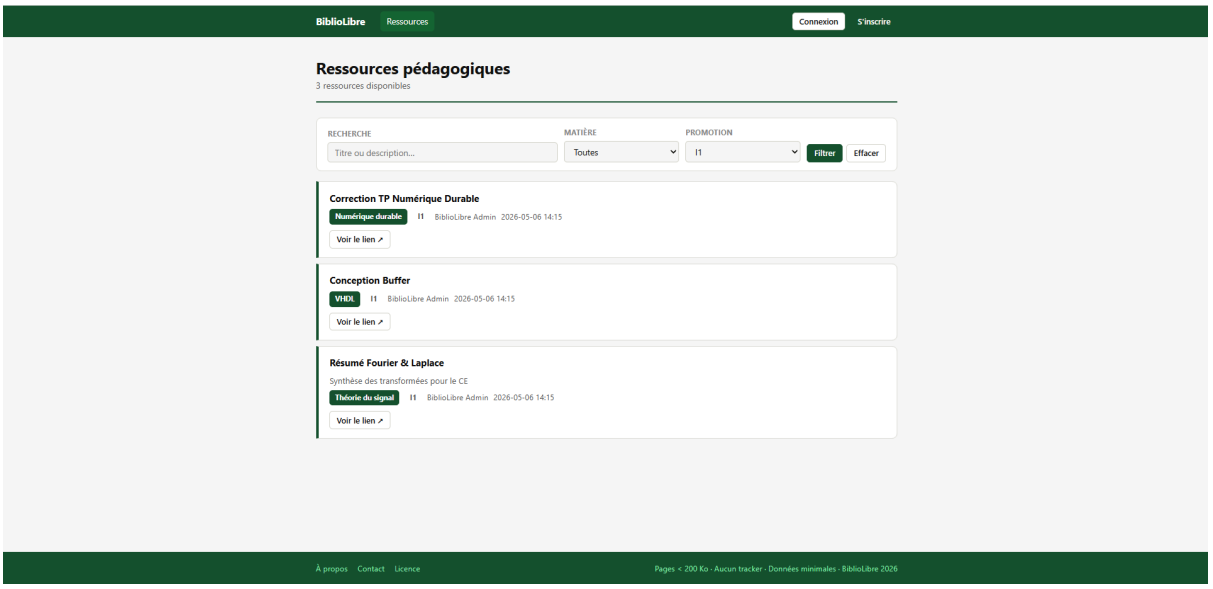
Backlog

Fonctionnalité	Priorité	Statut
Inscription / connexion / déconnexion	Must Have	Livré
Affichage public des ressources	Must Have	Livré
Dépôt d'une ressource par un utilisateur connecté	Must Have	Livré
Modification et suppression par l'auteur	Must Have	Livré
Filtrage par matière et promotion	Should Have	Livré
Interface d'administration des utilisateurs	Must Have	Livré
Page d'accueil avec bloc Green IT	Must Have	Livré
Modification du profil utilisateur	Should Have	Livré
Recherche plein-texte	Could Have	Non livré
Export CSV des ressources	Won't Have for now	Non livré

Captures d’écran du site



Page d’accueil



Page ressources avec un filtre par année (I1)

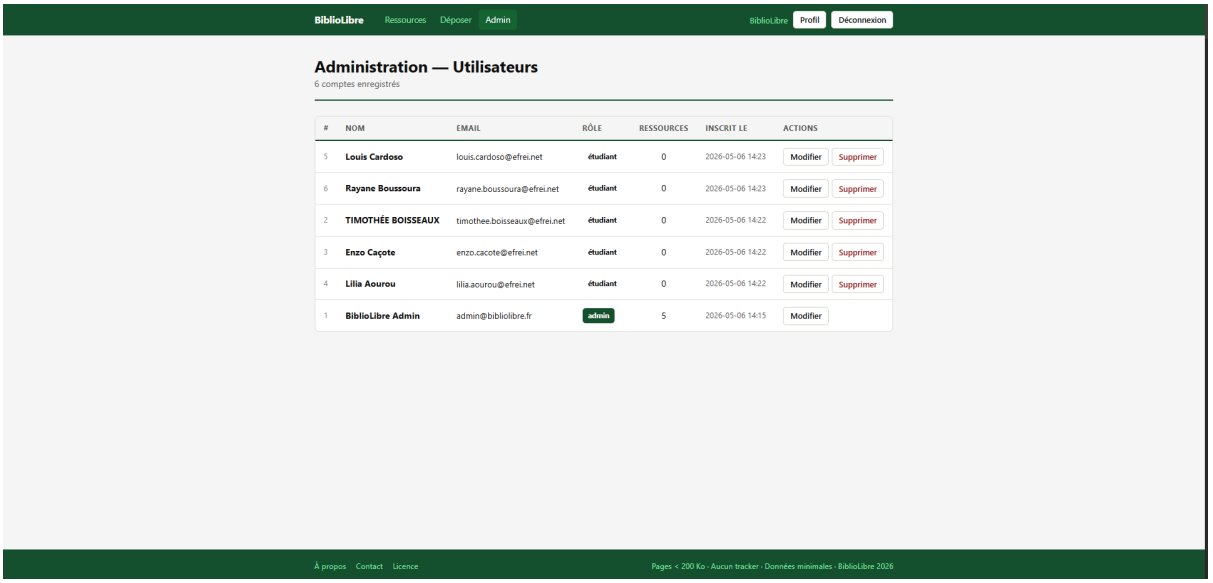


Tableau de bord de l’administrateur. Liste des comptes

Lien vers dépot GitHub et vers le site déployé

Dépôt GitHub : https://github.com/RayaneBsr/Projet-Green-IT

URL déployée : https://bibliolibre.onrender.com